

Proyecto

Infraestructura para la
Computación de Altas Prestaciones

Curso: 2025/2026

David Jávega Luis

Iván Bazar Herrera

Alberto Martínez Sánchez

ÍNDICE

Tarea 1	Error! Bookmark not defined.
Tarea 2	Error! Bookmark not defined.
Resumen de recursos y servicios desplegados.....	23

Plan de trabajo

1. Lo primero que hicimos fue diseñar el diagrama de la solución para tener claro los recursos que íbamos a necesitar. Esta tarea nos duró un par de días, tanto para entender correctamente el enunciado del proyecto como para elegir los recursos necesarios para la resolución.
2. Una vez que teníamos claro el diseño de la solución, decidimos empezar creando la infraestructura mediante la plantilla de CloudFormation. El fichero que usamos va adjunto a la entrega. Esto nos tardó un día de trabajo aproximadamente.
3. Después de crear la plantilla hicimos el pipeline de datos: Lambda, DynamoDB... Estuvimos unos 3 días aproximadamente para diseñar la función, probarla y mejorarla para que sea más tolerante a fallos.
4. Una vez creado el pipeline, nos pusimos con la tarea de la API REST. Esta fue la parte más complicada para nosotros, sobre todo para diseñar la función Flask y crear el contenedor. Fueron unos 3 días de trabajo para poder terminarlo y probarlo

Todas las tareas las hicimos los 3 miembros del grupo juntos.

Arquitectura de la solución

El diagrama de la solución que hemos propuesto es el siguiente:

Diagrama de arquitectura de AWS Cloud para un proyecto de IoT. El diagrama muestra la configuración de recursos en una región de AWS, incluyendo subredes públicas, grupos de seguridad, instancias de Docker, servicios ECS, un repositorio de código, un gateway de Internet, un grupo de seguridad de recursos, un equilibrador de carga, una instancia de ingestión de datos, una tabla de DynamoDB y una notificación de SNS. El flujo de datos comienza con un archivo CSV de temperaturas que se ingresa a un bucket de datos, luego se procesa y se almacena en una tabla de DynamoDB, finalmente se envía una notificación de SNS.

La infraestructura básica la hemos creado mediante una plantilla de CloudFormation con un archivo `.YAML`, ya que esto permite que sea un proyecto replicable. Esta plantilla despliega una VPC con la dirección `10.0.0.0/16` con la subred pública `10.0.1.0/24`. Dentro de esta subred está incluida la instancia del Docker y su grupo de seguridad `FlaskSecourityGroup`, además de un grupo de seguridad `WebServer`. Aquí se adjunta una foto de que la instancia se ha creado correctamente dentro de la subred pública de la vpc:

Infraestructura para la Computación de Altas Prestaciones

The screenshot shows the AWS Management Console for an EC2 instance. The instance is named 'proy-DockerServer' and has the ID 'i-01a35394e23662652'. It is in the 'En ejecución' (Running) state. The console displays various details including the public IP address (44.195.40.97), private IP address (10.0.1.54), DNS public (ec2-44-195-40-97.compute-1.amazonaws.com), and the instance type (t2.micro). It also shows the VPC (vpc-07a557322986a969f) and the subnet (subnet-0151f2772f53b381a). The instance is running the Amazon Linux 2 AMI.

Después de hacer esto, nos pusimos con el pipeline de datos. Para ello, lo primero que hicimos fue dividir el fichero de temperaturas en 4 ficheros de 100 filas cada uno, con el objetivo de probar después el correcto funcionamiento de la función lambda.

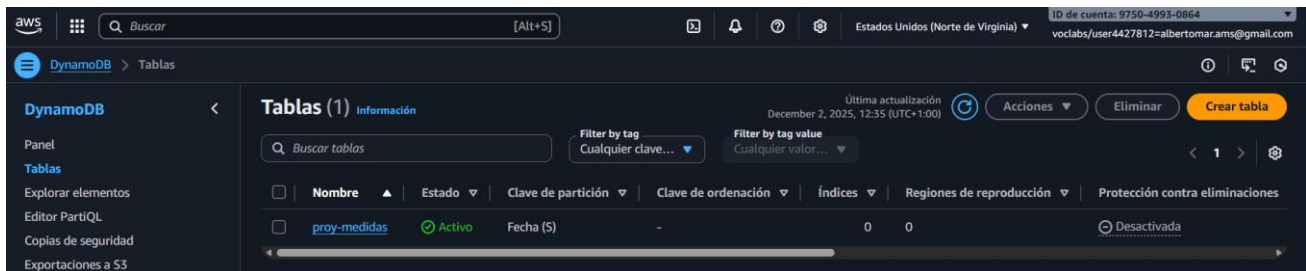
Después de hacer esto, creamos el bucket de S3 en el que iremos añadiendo los ficheros .csv de temperatura.

The screenshot shows the Amazon S3 console. It displays a list of buckets under the 'Buckets de uso general' tab. The buckets are:

Nombre	Región de AWS	Fecha de creación
cf-templates-1lqd8bmx2q0b-us-east-1	EE.UU. Este (Norte de Virginia) us-east-1	19 Nov 2025 6:12:35 PM CET
proy-ingesta-datos-975049930864	EE.UU. Este (Norte de Virginia) us-east-1	20 Nov 2025 5:12:06 PM CET

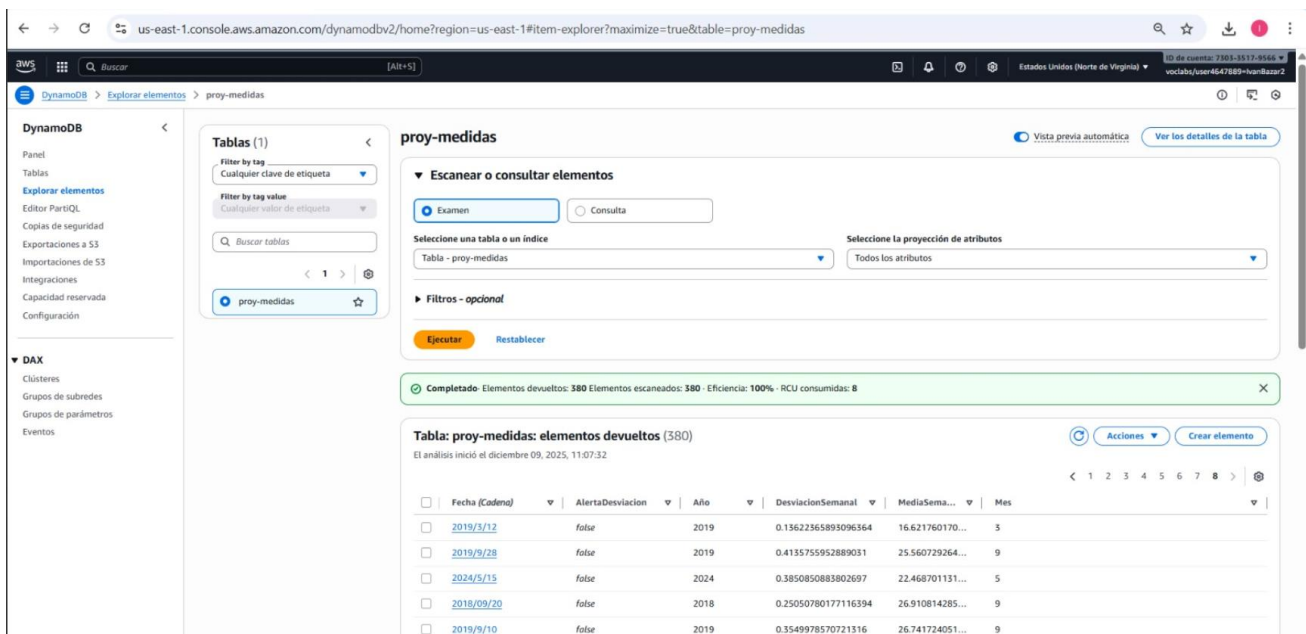
Infraestructura para la Computación de Altas Prestaciones

Una vez creado el Bucket, creamos la tabla DynamoDB donde guardaríamos los datos procesados por la función lambda.



Por último, la creación de la función lambda. Dentro de esta función, se añade la notificación de eventos y la separación de la fecha en mes y año. Para probar la notificación de eventos, primero añadimos un correo electrónico personal, así nos aseguramos de que funcionaba correctamente. La función lambda para la solución viene adjunta a esta entrega con las explicaciones pertinentes.

Para comprobar el correcto funcionamiento de la función, fuimos añadiendo los ficheros que dividimos al bucket poco a poco y comprobando que la tabla dynamo añadía correctamente los datos y con el formato necesario.



API Rest

A continuación, se describen los pasos seguidos para la creación de la API Rest (*Todos los comandos para realizar los pasos que se describen a continuación están especificados en orden en el fichero adjunto a la entrega).

Lo primero de todo era conectarse a la CLI de AWS. Esto se hizo mediante ssh. Una vez dentro, lo siguiente que hicimos fue crear un directorio llamado `proy-flask_docker`. Dentro de él, creamos el fichero con el servicio Flask. El contenido de este fichero también va adjunto a la entrega. Este fichero Python se encargaba de calcular los 3 datos requeridos en el enunciado de este proyecto. Después, el fichero `requirements.txt` y el fichero `Dockerfile`, ambos también dentro del directorio creado.

Ahora es el momento de crear la imagen del contenedor. Los pasos a seguir son: primero crear el contenedor `proy-flask_container`, comprobar que se ha creado la imagen, ejecutar la instancia de la imagen Docker y comprobar que el contenedor aparece en el listado de contenedores ejecutándose.

Por último, habría que comprobar que el servicio se lanzaba correctamente poniendo la dirección IPv4 de la instancia Docker en el buscador, usando una de los 3 servicios creados: `/maxdiff`, `/sd` y `/temp`. Esto también se puede hacer desde la CLI con un comando GET.

```
← → ↻ ⚠ No es seguro 98.81.120.188:5000/maxdiff?month=2&year=2019
Dar formato al texto ☐
{"valor":1.5291385803254887}
```

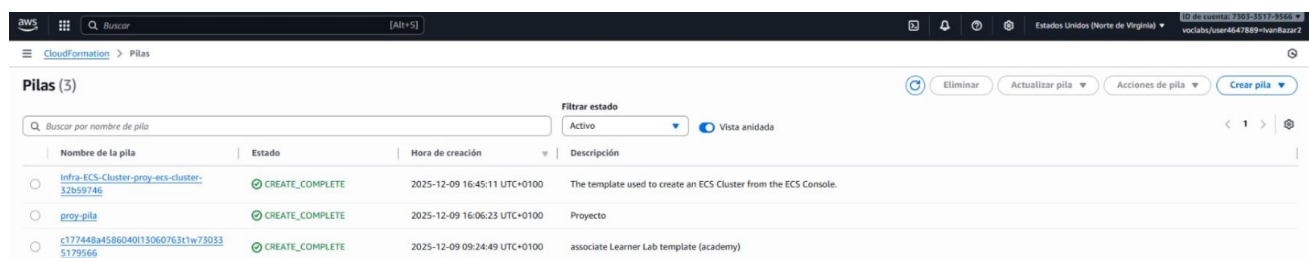
```
← → ↻ ⚠ No es seguro 98.81.120.188:5000/temp?month=2&year=2019
Dar formato al texto ☐
{"valor":12.762877253078562}
```

```
← → ↻ ⚠ No es seguro 98.81.120.188:5000/sd?month=2&year=2019
Dar formato al texto ☐
{"valor":0.3211802255842497}
```

Escalabilidad

Con el objetivo de que el proyecto sea escalable, vamos a crear un balanceador de carga. Para ello, es necesario crear una serie de servicios antes. Lo primero que vamos a hacer es crear un Repositorio de ECR para almacenar nuestra imagen Docker creada anteriormente. Para ello, debemos escribir los comandos que nos aparecen al hacer click en “Ver comandos de envío”.

A continuación, creamos un clúster de ECS donde ejecutaremos nuestro servicio. Lo haremos configurándolo con un mínimo de 2 instancias y un máximo de 4 instancias. Además, tendremos que conectarlo con la VPC, redes y SG creados en la plantilla de CloudFormation. Una vez creado, nos debe de aparecer una nueva pila:



Nombre de la pila	Estado	Hora de creación	Descripción
infra-ECS-Cluster-proy-ecs-cluster-32b59746	CREATE_COMPLETE	2025-12-09 16:45:11 UTC+0100	The template used to create an ECS Cluster from the ECS Console.
proy-pila	CREATE_COMPLETE	2025-12-09 16:06:23 UTC+0100	Proyecto
c177448a4586040130607631w730335179566	CREATE_COMPLETE	2025-12-09 09:24:49 UTC+0100	associate Learner Lab template (academy)

Después, vamos a crear una nueva tarea ECS. Dentro de esta tarea, además, se creará un contenedor en el que añadiremos nuestra imagen del repositorio de ECR. Este contenedor tendrá puerto 5000 y protocolo HTTP.

Una vez creada nuestra tarea, tendremos que crear nuestro balanceado de carga, que nos va a permitir que se distribuya el tráfico entre instancia EC2, el contenedor y direcciones IP en nuestras dos zonas de disponibilidad. Dentro de este balanceador tendremos que crear un grupo de destino para añadirlo como agente de escucha. El umbral en buen y mal estado será 2, esto es, el destino se considera en buen estado si recibe dos comprobaciones de estado correctas.

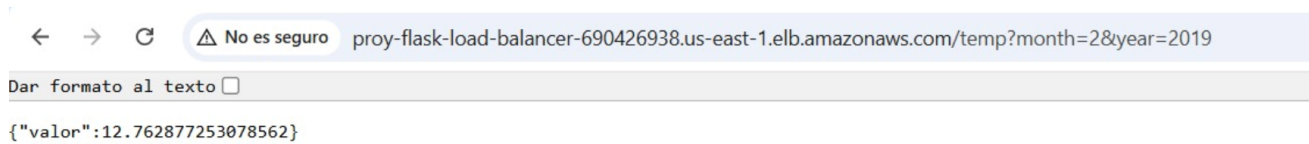
Por último, crearemos el servicio de ECS para poder desplegar la aplicación Flask. El clúster en el que pongamos este servicio será el que hemos creado antes. Pondremos 2 tareas deseadas y añadiremos el balanceador como equilibrio de carga.

Al configurar cada uno de estos servicios, hemos tenido que añadir la VPC y la subred creadas en la plantilla de CloudFormation. Además, tanto para el clúster como para el servicio configuraremos como grupo de seguridad el “Flask Secourity Group”, mientras que para el balanceador añadimos el grupo “BalanceadorSecourityGroup”.

Para que el servicio funcione cuando copiamos el nombre DNS del balanceador de carga, debemos de crear antes un punto de enlace, el cual llamaremos *proy-endpoint*. Elegiremos

Infraestructura para la Computación de Altas Prestaciones

nuestra VPC, nuestras subredes y las tablas de enrutamiento. Comprobamos que, efectivamente, el servicio funciona:



GUÍA DE REPLICACIÓN

Crear infraestructura:

1. Lo primero que hay que hacer es crear un par de claves. En el apartado EC2/Red y seguridad/Pares de Claves. Lo llamamos *proy-docker-keypair*
2. Ahora nos vamos a CloudFormation y le damos a “Crear Pila” y “con recursos estándar”
3. Elegimos la opción cargar archivo existente y añadimos el fichero `proy-infraestructura.yaml`. El fichero tiene el siguiente contenido:

```
AWSTemplateFormatVersion: 2010-09-09
```

```
Description: >-
```

```
Proyecto
```

```
# This template creates:
```

```
# VPC
```

```
# Internet Gateway
```

```
# Public Route Table
```

```
# Public Subnet1
```

```
# Public Subnet2
```

```
# EC2 instance
```

```
# Bucket S3
```

```
#####
```

```
# Resources section
```

```
#####
```

```
Resources:
```

Infraestructura para la Computación de Altas Prestaciones

```
## VPC
VPC:
  Type: AWS::EC2::VPC
  Properties:
    EnableDnsSupport: true
    EnableDnsHostnames: true
    CidrBlock: 10.0.0.0/16
    Tags:
      - Key: Name
        Value: proy-VPC

## Internet Gateway
InternetGateway:
  Type: AWS::EC2::InternetGateway

VPCGatewayAttachment:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref VPC
    InternetGatewayId: !Ref InternetGateway

## Public Route Table
PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC

PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: VPCGatewayAttachment
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

## Public Subnet 1
PublicSubnet1:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 10.0.1.0/24
    AvailabilityZone: us-east-1a
    MapPublicIpOnLaunch: true
    Tags:
```

Infraestructura para la Computación de Altas Prestaciones

```
- Key: Name
  Value: proy-pulic-subnet-1

## Public Subnet 2
PublicSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 10.0.3.0/24
    AvailabilityZone: us-east-1b
    MapPublicIpOnLaunch: true
    Tags:
      - Key: Name
        Value: proy-public-subnet-2

PublicSubnet1RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet1
    RouteTableId: !Ref PublicRouteTable

PublicSubnet2RouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet2
    RouteTableId: !Ref PublicRouteTable

# Grupo de seguridad para el balanceador
BalanceadorSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Enable HTTP ingress
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 80
        ToPort: 80
        CidrIp: 0.0.0.0/0
    Tags:
      - Key: Name
        Value: BalanceadorSecurityGroup

# Instancia Docker
DockerInstance:
  Type: AWS::EC2::Instance
```

Infraestructura para la Computación de Altas Prestaciones

```
Properties:
  InstanceType: t2.micro
  ImageId: ami-08a0d1e16fc3f61ea
  KeyName: proy-docker-keypair
  IamInstanceProfile: LabInstanceProfile
  NetworkInterfaces:
    - GroupSet:
        - !Ref DockerInstanceSecurityGroup
      AssociatePublicIpAddress: true
      DeviceIndex: 0
      DeleteOnTermination: true
      SubnetId: !Ref PublicSubnet1
  Tags:
    - Key: Name
      Value: proy-DockerServer
  UserData:
    Fn::Base64: !Sub |
      #!/bin/bash
      dnf update -y
      #install Docker
      dnf -y install docker

DiskVolume:
  Type: AWS::EC2::Volume
  Properties:
    Size: 100
    AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
  DeletionPolicy: Snapshot

DiskMountPoint:
  Type: AWS::EC2::VolumeAttachment
  Properties:
    InstanceId: !Ref DockerInstance
    VolumeId: !Ref DiskVolume
    Device: /dev/sdh

DockerInstanceSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Enable TCP ingress for Flask
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
```

Infraestructura para la Computación de Altas Prestaciones

```
    FromPort: 5000
    ToPort: 5000
    CidrIp: 0.0.0.0/0
  - IpProtocol: tcp
    FromPort: 22
    ToPort: 22
    CidrIp: 0.0.0.0/0
  Tags:
    - Key: Name
      Value: Flask Security Group

## Bucket S3 para Ingesta de Datos
S3BucketIngesta:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: !Sub "proy-ingesta-datos-${AWS::AccountId}"
    PublicAccessBlockConfiguration:
      BlockPublicAcls: true
      BlockPublicPolicy: true
      IgnorePublicAcls: true
      RestrictPublicBuckets: true
  Tags:
    - Key: Name
      Value: proy-ingesta-datos
#####
# Outputs section
#####

Outputs:

PublicSubnet:
  Description: The subnet ID to use for public web servers
  Value: !Ref PublicSubnet1
  Export:
    Name: !Sub '${AWS::StackName}-SubnetID'

VPC:
  Description: VPC ID
  Value: !Ref VPC
  Export:
    Name: !Sub '${AWS::StackName}-VPCID'

BucketName:
  Description: Bucket ID
  Value: !Ref S3BucketIngesta
```

Infraestructura para la Computación de Altas Prestaciones

```
Export:
```

```
Name: !Sub '${AWS::StackName}-BucketName'
```

5. En nombre, ponemos *proy-pila*
6. En el apartado configurar opciones de pila, elegimos el rol *LabRole*
7. Le damos a enviar y esperamos a que el estado de la pila esté en CREATE_COMPLETE

Pipeline de datos:

1. DynamoDB:
 - a. Empezamos creando la tabla **DynamoDB**. En el apartado tablas le damos a Crear Tabla. Le ponemos de nombre *proy-medidas* y clave de partición: fecha.
2. Función Lambda:
 - a. Ahora vamos con la función **lambda**. Le damos a crear función y le añadimos de nombre *proy-ingesta*
 - b. En versión ejecutable, seleccionamos python3.14; y en el apartado Rol de ejecución, seleccionamos “uso de rol existente” y elegimos *LabRole* y le damos a Crear Función

Dentro de la función, escribimos el código de este fichero `lambda_ingesta.py`. El contenido del fichero es el siguiente:

```
import json
import urllib.parse
import boto3
import csv
from datetime import datetime
from decimal import Decimal

# Recursos de AWS
s3 = boto3.resource('s3')
dynamodb = boto3.resource('dynamodb')

# Tabla de DynamoDB donde guardaremos los datos semanales
temperatura_table = dynamodb.Table('proy-medidas')

def lambda_handler(event, context):

    print("Evento recibido por la función Lambda:")
    print(json.dumps(event, indent=2, ensure_ascii=False))

    # Obtener bucket y key del objeto subido a S3
    bucket = event['Records'][0]['s3']['bucket']['name']
```

Infraestructura para la Computación de Altas Prestaciones

```

key = urllib.parse.unquote_plus(
    event['Records'][0]['s3']['object']['key'],
    encoding='utf-8'
)

# Fichero temporal donde se descargará el CSV en la Lambda
local_filename = '/tmp/proy-datos_semanales.csv'

# Descargar el fichero desde S3
try:
    s3.meta.client.download_file(bucket, key, local_filename)
    print(f"Fichero descargado de S3: s3://{bucket}/{key} ->
{local_filename}")
except Exception as e:
    print(e)
    print(
        f'Error obteniendo el objeto {key} del bucket {bucket}. '
        'Comprueba que existen y que están en la misma región que la
función.'
    )
    raise e

# Leer el CSV y escribir cada fila en DynamoDB
row_count = 0

with open(local_filename, newline='', encoding='utf-8') as csvfile:
    reader = csv.DictReader(csvfile, delimiter=',')

    for row in reader:
        row_count += 1

        try:
            fecha_str = row['Fecha']
            media = float(row['Medias'])
            sd = float(row['Desviaciones'])

            # Extraer mes y año de la fecha (para las estadísticas
mensuales)

            fecha_dt = datetime.strptime(fecha_str, '%Y/%m/%d')
            año = fecha_dt.year
            mes = fecha_dt.month

            # Alerta de desviación
            alerta_desviacion = sd > 0.5

```

Infraestructura para la Computación de Altas Prestaciones

```
        if alerta_desviacion:
            message = f"Alerta de desviación en la fecha
{fecha_str}: {sd}"
            print(message)
            sns = boto3.client('sns')
            alertTopic = "proy-alerta"
            snsTopicArn = [t['TopicArn'] for t in
sns.list_topics()['Topics'] if t['TopicArn'].lower().endswith(':' +
alertTopic.lower())][0]
            sns.publish(
                TopicArn=snsTopicArn,
                Message=message,
                Subject='Alerta de desviación',
                MessageStructure='string'
            )

        # Item que insertaremos en DynamoDB
        item = {
            'Fecha': fecha_str,                # PK de la tabla
            'MediaSemanal': Decimal(str(media)),    #
temperatura media de la semana
            'DesviacionSemanal': Decimal(str(sd)), # desviación
estándar de la semana
            'Año': año,
            'Mes': mes,
            'AlertaDesviacion': alerta_desviacion
        }

        # Insertar en DynamoDB
        temperatura_table.put_item(Item=item)

    except Exception as e:
        print(f"Error en la Fila {row_count} ---")
        print(f"Error: {e}")

        raise e
```

- c. Una vez creada la función, tendremos que editar el tiempo de ejecución para que no haya errores debidos a falta de tiempo o espacio. En el apartado configuración, le damos a editar y ponemos 10s de tiempo de espera y 256MB de memoria.
3. Notificación de eventos:
 - a. Antes de añadir el archivo al bucket, es necesario crear la **notificación de eventos**. Nos vamos al bucket de S3 llamado *proy-ingesta-datos* y, en

Infraestructura para la Computación de Altas Prestaciones

- propiedades, buscamos “Crear notificación de eventos”. La llamamos proy-notificación. En tipos de evento, seleccionamos “Todos los eventos de creación de objetos”. En destino, función lambda y seleccionamos la que acabamos de crear.
- b. Ahora nos vamos a Amazon SNS. En apartado temas, le damos a “Crear un tema”. Nombre: proy-alerta. En detalles, seleccionamos estándar y le damos a crear un tema.
 - c. Dentro de temas, seleccionamos el tema que acabamos de crear y le damos a “Crear una suscripción”. Donde pone protocolo, seleccionamos correo electrónico y añadimos el correo victoria.beltran@upct.es y le damos a crear.
4. Añadir ficheros:
- a. Nos vamos a nuestro Bucket de S3 y añadimos los ficheros de temperatura divididos de uno en uno para que la tabla se vaya rellenando. Comprobamos que la tabla se ha rellenado correctamente

Api rest:

1. Lo primero es abrir la terminal desde el repositorio donde está nuestro keypair. Una vez dentro, escribimos el siguiente comando para acceder a nuestra instancia: “`ssh -i proy-docker-keypair.pem ec2-user@<IPv4_instancia_docker>`”
2. Ahora vamos a crear los archivos necesarios. Escribimos los siguientes comandos:
 - a. Carpeta donde estará el docker: `mkdir proy-flask_docker`
 - b. Nos metemos a la carpeta y creamos el archivo requirements: `nano proy-requirements.txt`. Dentro del archivo escribimos lo siguiente:

```
Modified
Flask==1.1.2
Werkzeug==1.0.1
Jinja2==2.11.3
MarkupSafe==1.1.1
itsdangerous==1.1.0
click==7.1.2
boto3
```

- c. Archivo con el código fuente: `nano proy-app.py`. El contenido de este fichero es el siguiente

```
from flask import Flask, request, jsonify
import boto3
from boto3.dynamodb.conditions import Attr
```

Infraestructura para la Computación de Altas Prestaciones

```
app = Flask(__name__)

# Conexión a la base de datos
db = boto3.resource('dynamodb', region_name='us-east-1')
tabla = db.Table('proy-medidas')

# Función auxiliar para no repetir código
def buscar(mes, anio):
    filtro = Attr('Mes').eq(int(mes)) & Attr('Año').eq(int(anio))
    return tabla.scan(FilterExpression=filtro)['Items']

@app.route('/')
def inicio():
    return "<h1>Servidor Web Activo</h1>"

# Temperatura Media (/temp)
@app.route('/temp')
def media():
    m = request.args.get('month')
    a = request.args.get('year')

    datos = buscar(m, a)
    if len(datos) == 0:
        return "No hay datos"

    suma = 0
    for d in datos:
        suma += float(d['MediaSemanal'])

    promedio = suma / len(datos)
    return jsonify({'valor': promedio})

# Desviación Máxima (/sd)
@app.route('/sd')
def desviacion():
    m = request.args.get('month')
    a = request.args.get('year')

    datos = buscar(m, a)

    maximo = 0
    for d in datos:
        valor = float(d['DesviacionSemanal'])
        if valor > maximo:
            maximo = valor
```

Infraestructura para la Computación de Altas Prestaciones

```
    return jsonify({'valor': maximo})

# Diferencia con mes anterior (/maxdiff)
@app.route('/maxdiff')
def diferencia():
    m = int(request.args.get('month'))
    a = int(request.args.get('year'))

    # Máximo de este mes
    datos_hoy = buscar(m, a)
    max_hoy = 0
    for d in datos_hoy:
        if float(d['MediaSemanal']) > max_hoy:
            max_hoy = float(d['MediaSemanal'])

    # Mes anterior
    if m == 1:
        m_ant, a_ant = 12, a - 1
    else:
        m_ant, a_ant = m - 1, a

    datos_ant = buscar(m_ant, a_ant)
    max_ant = 0
    for d in datos_ant:
        if float(d['MediaSemanal']) > max_ant:
            max_ant = float(d['MediaSemanal'])

    resultado = max_hoy - max_ant
    return jsonify({'valor': resultado})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Irá adjunto a la entrega

- c. Archivo Dockerfile: *nano Dockerfile*. En este fichero ponemos lo siguiente:

```
# syntax=docker/dockerfile:1.4
FROM python:3.8-alpine
WORKDIR /app
COPY proy-requirements.txt proy-requirements.txt
RUN pip3 install -r proy-requirements.txt
COPY . .
CMD ["python3", "proy-app.py"]
```

Infraestructura para la Computación de Altas Prestaciones

2. Creamos la imagen del contenedor

- Inicio el docker: `sudo systemctl start docker`
- Crear la imagen del contenedor: `sudo docker build -t proy-flask_container .`
- Ejecutar el docker: `sudo docker run -p 5000:5000 -d proy-flask_container`
- Comprobamos que la imagen se ha creado y está en ejecución: `sudo docker ps`

3. Ejecutar la API Rest

- `TOKEN=$(curl --request PUT "http://169.254.169.254/latest/api/token" --header "X-aws-ec2-metadata-token-ttl-seconds: 21600")`
- Comprobamos la dirección IPv4 de la instancia: `curl http://169.254.169.254/latest/meta-data/public-ipv4 --header "X-aws-ec2-metadata-token: $TOKEN"`
- Comprobamos que el servicio web está activo: `curl http://<direccion_salida_comando_anterior>:5000`



No es seguro

98.81.120.188:5000

Servidor Web Activo

Escalabilidad:

1. Repositorio ECR

- Nos vamos a la pestaña ECR y le damos a “Crear Repositorio”, poniéndole de nombre proy-repo
- Ahora, le damos a “Ver Comandos de envío”. Posteriormente, abrimos de nuevo nuestra terminal de AWS y escribimos los comandos que nos pone en orden
- `sudo aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin <n_de_cuenta>.dkr.ecr.us-east-1.amazonaws.com`
- `sudo docker tag proy-flask_container <n_de_cuenta>.dkr.ecr.us-east-1.amazonaws.com/proy-repo:proy-flask_container`
- `sudo docker push <n_de_cuenta>.dkr.ecr.us-east-1.amazonaws.com/proy-repo:proy-flask_container`

Infraestructura para la Computación de Altas Prestaciones

2. Clúster ECS

- a. Nos vamos a la pestaña ECS y, en clústeres, le damos a “Crear Clúster”. Le ponemos de nombre *proy-ecs-cluster*
- b. En el apartado infraestructura avanzada, seleccionamos la opción “Fargate e instancias autoadministradas”. En modelo de aprovisionamiento seleccionamos “bajo demanda”. Tipo de instancia: t2.micro. En capacidad de demanda ponemos mínimo 2 instancias y máximo 4. Añadimos también el par de claves creado en el primer punto.
- c. Abrimos la pestaña “Configuración de redes”. Dentro de ella, seleccionamos nuestra VPC y nuestras dos subredes creadas. Además, añadimos el Grupo de seguridad DockerInstance, llamado *Flask Secourity Group*, y quitamos el que estaba puesto por defecto.
- d. Por último, en “Etiquetas”, añadimos clave: cluster-ecs y valor: proy.

3. Tarea ECS

- a. Nos vamos a Definiciones de tareas, dentro de ECS, y le damos a “Crear una nueva definición de tarea”. De nombre le ponemos *proy-task*.
- b. En requisitos de infraestructura, seleccionamos “Instancias de Amazon EC2”. En CPU **.2** vCPU y, en memoria, **.3** GB. Añadimos también nuestro LabRole en los roles de tarea y de ejecución de tarea.
- c. Dentro de la pestaña Contenedor vamos a crear nuestro contenedor. De nombre le pondremos *proy-container*. Le añadimos la URI de la imagen del contenedor que hemos creado anteriormente en nuestro repositorio de ECR. Puerto del contenedor: 5000, protocolo: TCP, nombre del puerto: *proy-flasktraffic* y protocolo de app: HTTP. Configuraremos también lo siguiente: **.1** vCPU, **1** GPU, **.3** GB estrictos, **.1** GB flexibles.

4. Balanceador de carga

- a. En la pestaña EC2, nos vamos a balanceadores de carga y hacemos click en “Crear balanceador de carga”. Elegimos Crear balanceador de carga de **aplicaciones**. Le ponemos el nombre *proy-flask-load-balancer*
- b. En mapeo de red, añadimos nuestra vpc, las dos zonas de disponibilidad (1a y 1b) con sus dos respectivas subredes públicas. En el grupo de seguridad, añadimos nuestro grupo de seguridad “BalanceadorSecourityGroup”, quitando también el que venía por defecto
- c. En Agentes de escucha y direccionamiento configuramos el puerto 80 y le damos a “crear un grupo de destino”
- d. Esto nos abrirá una nueva pestaña donde crearemos un grupo de destino. Vamos a elegir el tipo de destino “Direcciones IP”. De nombre le pondremos *proy-ip-target-group*. Añadimos el protocolo HTTP con puerto 5000. Añadimos nuestra vpc. En comprobaciones de estado elegiremos la siguiente configuración: umbral en buen estado: **2**, en mal estado: **2**, intervalo **6s**. Le damos a crear un grupo de destino

Infraestructura para la Computación de Altas Prestaciones

- e. Ahora volvemos a nuestra pestaña de creación del balanceador de carga y añadimos el grupo de destino que acabamos de crear. Después de esto, le damos a “Crear balanceador de carga”
5. Implementación del servicio ECS
 - a. Nos vamos de nuevo a ECS. En definiciones de tarea seleccionamos nuestra tarea *proy-task*, y, en implementar, hacemos click en “Crear servicio”. En nombre, ponemos *proy-ecs-service*
 - b. En clúster existente seleccionamos nuestro cluster *proy-ecs-cluster*. El tipo de servicio será REPLICA. Pondremos 2 tareas deseadas.
 - c. Abrimos el desplegable Redes y dejamos nuestras vpc y subredes seleccionadas. El grupo de seguridad será el DockerInstance (Flask Security Group)
 - d. Ahora nos vamos a Equilibrio de carga y seleccionamos la casilla “Usar un equilibrio de carga”. En tipo, elegimos “Balanceador de carga de aplicaciones”. El contenedor será el *proy-container*. Le damos a “Usar un balanceador de carga existente” y seleccionamos el que acabamos de crear *proy-flask-load-balancer*. En agente de escucha, seleccionamos “Utilizar un agente de escucha existente” y añadimos el HTTP:80. En grupo de destino, le damos click a “Utilizar un grupo de destino existente” y seleccionamos nuestro *proy-ip-target-group*. Le damos a Crear
6. Punto de enlace
 - a. Vamos a crear ahora el punto de enlace para que se comuniquen los servicios y la instancia correctamente. Nos vamos a VPC y, en puntos de conexión, le damos a “Crear punto de enlace”. De nombre pondremos *proy-endpoint*.
 - b. En la pestaña servicios, vamos a elegir dynamodb de tipo Gateway
 - c. En configuración de red, elegimos nuestra VPC y las dos tablas de enrutamiento de las 2 subredes creadas. Por último, le damos a crear y comprobamos
7. Copiamos el nombre DNS del balanceador de carga en el buscador y comprobamos que el servicio funciona correctamente. Con esto, habríamos terminado los requisitos del proyecto.

Resumen de recursos y servicios desplegados

Identificador de recurso	Nombre de recurso	Tipo de recurso en AWS	Comentario (opcional)
vpc-03dfc4fa50526a985	Proy-VPC	VPC	VPC creada en la plantilla donde incluiremos la subred y la instancia
subnet-0f761470e90653b56	Proy-public-subnet-1	Subnet	Subred pública donde se incluirá la instancia Docker
subnet-08d280b82939e6dba	Proy-public-subnet-2	Subnet	Segunda subnet pública que se usará para cumplir el requisito de escalabilidad
sg-0ddda48cc40052bb6	Flask Secourity Group	Grupo de Seguridad Docker Server	Grupo de seguridad que se utilizará para la instancia docker
sg-0639de1bd6e794c04	Balanceador Secourity Group	Grupo de Seguridad Balanceador	Grupo de seguridad que se utilizará para el balanceador de carga
i-078363c17b45b5de	Proy-DockerServer	Instancia EC2	Instancia Docker creada en la plantilla donde se incluirá el servicio Flask
	Proy-ingesta-datos	Bucket S3	Bucket donde se añadirán los ficheros para la ejecución de la función lambda
	Proy-ingesta	Lambda	Función que manda la notificación y

Infraestructura para la Computación de Altas Prestaciones

			añade los datos a la tabla dynamo
	Proy-medidas	DynamoDB	Tabla donde se almacenan los datos procesados para la posterior ejecución de la app Flask
	Proy-repo	Repositorio ECR	Repositorio donde se almacena la imagen del contenedor
	Proy-ecs-cluster	Cluster ECS	Clúster con el grupo de instancias donde ejecutaremos nuestra aplicación
	Proy-task	Tarea ECS	
	Proy-ecs-service	Servicio ECS	Servicio que garantiza que la tarea se esté ejecutando
	Proy-load-balancer	Balanceador de Carga	